```
!wget -O content.jpg https://pytorch.org/tutorials/_static/img/neural-style/dancing.jpg
!wget -O style.jpg https://pytorch.org/tutorials/_static/img/neural-style/picasso.jpg
```

```
--2026-02-27 05:24:04--  https://pytorch.org/tutorials/_static/img/neural-style/dancing.jpg
Resolving pytorch.org (pytorch.org)... 23.185.0.2, 2620:12a:8000::2, 2620:12a:8001::2
Connecting to pytorch.org (pytorch.org)|23.185.0.2|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://docs.pytorch.org/tutorials/_static/img/neural-style/dancing.jpg [following]
--2026-02-27 05:24:05--  https://docs.pytorch.org/tutorials/_static/img/neural-style/dancing.jpg
Resolving docs.pytorch.org (docs.pytorch.org)... 185.199.108.153, 185.199.109.153, 185.199.110.153, ...
Connecting to docs.pytorch.org (docs.pytorch.org)|185.199.108.153|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 40484 (40K) [image/jpeg]
Saving to: ‘content.jpg’

content.jpg         100%[===================>]  39.54K  --.-KB/s    in 0.005s

2026-02-27 05:24:05 (7.73 MB/s) - ‘content.jpg’ saved [40484/40484]

--2026-02-27 05:24:05--  https://pytorch.org/tutorials/_static/img/neural-style/picasso.jpg
Resolving pytorch.org (pytorch.org)... 23.185.0.2, 2620:12a:8000::2, 2620:12a:8001::2
Connecting to pytorch.org (pytorch.org)|23.185.0.2|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://docs.pytorch.org/tutorials/_static/img/neural-style/picasso.jpg [following]
--2026-02-27 05:24:05--  https://docs.pytorch.org/tutorials/_static/img/neural-style/picasso.jpg
Resolving docs.pytorch.org (docs.pytorch.org)... 185.199.108.153, 185.199.109.153, 185.199.110.153, ...
Connecting to docs.pytorch.org (docs.pytorch.org)|185.199.108.153|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 114573 (112K) [image/jpeg]
Saving to: ‘style.jpg’

style.jpg           100%[===================>] 111.89K  --.-KB/s    in 0.02s

2026-02-27 05:24:05 (5.96 MB/s) - ‘style.jpg’ saved [114573/114573]
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, transforms
from torchvision.utils import save_image
from PIL import Image
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

image_size = 256

loader = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor()
])

def load_image(path):
    image = Image.open(path).convert("RGB")
    image = loader(image).unsqueeze(0)
    return image.to(device)

# Now these files exist
content_img = load_image("content.jpg")
style_img = load_image("style.jpg")

print("Images loaded successfully.")
```

```
Images loaded successfully.
```

```
# ============================================
# IMPROVED NEURAL STYLE TRANSFER (ONE CELL)
# ============================================

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import files

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
# ----------------------------
# Upload Content + Style Images
# ----------------------------
print("Upload CONTENT image")
uploaded_content = files.upload()
content_filename = list(uploaded_content.keys())[0]

print("Upload STYLE image (preferably a painting)")
uploaded_style = files.upload()
style_filename = list(uploaded_style.keys())[0]

# ----------------------------
# Image Settings
# ----------------------------
image_size = 512

loader = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor()
])

# ImageNet normalization (CRUCIAL)
normalize = transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)

def load_image(path):
    image = Image.open(path).convert("RGB")
    image = loader(image).unsqueeze(0)
    return image.to(device)

content_img = load_image(content_filename)
style_img = load_image(style_filename)

# ----------------------------
# Load VGG19
# ----------------------------
vgg = models.vgg19(pretrained=True).features.to(device).eval()

for param in vgg.parameters():
    param.requires_grad = False

# ----------------------------
# Layer Selection
# ----------------------------
content_layer = '21'  # conv4_2
style_layers = ['0', '5', '10', '19', '28']

# ----------------------------
# Feature Extraction
# ----------------------------
def get_features(image, model):
    features = {}
    x = normalize(image.clone())
    for name, layer in model._modules.items():
        x = layer(x)
        if name in style_layers:
            features[name] = x
        if name == content_layer:
            features[name] = x
    return features

content_features = get_features(content_img, vgg)
style_features = get_features(style_img, vgg)

# ----------------------------
# Gram Matrix
# ----------------------------
def gram_matrix(tensor):
    b, c, h, w = tensor.size()
    tensor = tensor.view(c, h * w)
    gram = torch.mm(tensor, tensor.t())
    return gram

style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_layers}

# ----------------------------
# Initialize Target
# ----------------------------
target = content_img.clone().requires_grad_(True).to(device)
```

```
content_weight = 1e4
style_weight = 1e3

optimizer = optim.Adam([target], lr=0.003)

# ----------------------------
# Optimization
# ----------------------------
steps = 800

for step in range(steps):

    target_features = get_features(target, vgg)

    # Content Loss
    content_loss = torch.mean(
        (target_features[content_layer] - content_features[content_layer]) ** 2
    )

    # Style Loss
    style_loss = 0
    for layer in style_layers:
        target_gram = gram_matrix(target_features[layer])
        style_gram = style_grams[layer]

        _, c, h, w = target_features[layer].shape
        layer_style_loss = torch.mean((target_gram - style_gram) ** 2)
        style_loss += layer_style_loss / (c * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    # Keep image values in valid range
    target.data.clamp_(0, 1)

    if step % 100 == 0:
        print(f"Step {step}, Total Loss: {total_loss.item()}")

# ----------------------------
# Display Result
# ----------------------------
def imshow(tensor):
    image = tensor.clone().detach().cpu().squeeze(0)
    image = transforms.ToPILImage()(image)
    plt.figure(figsize=(8,8))
    plt.imshow(image)
    plt.axis("off")
    plt.show()

print("Final Stylized Image:")
imshow(target)
```

```
Using device: cuda
Upload CONTENT image
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2026-02-27 at 11.07.25 am.png to Screenshot 2026-02-27 at 11.07.25 am.png
Upload STYLE image (preferably a painting)
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2026-02-27 at 11.06.57 am.png to Screenshot 2026-02-27 at 11.06.57 am.png
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretraine
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a w
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg
100%|██████████| 548M/548M [00:06<00:00, 94.4MB/s]
Step 0, Total Loss: 188473.3125
Step 100, Total Loss: 53961.46484375
Step 200, Total Loss: 40073.99609375
Step 300, Total Loss: 33826.1953125
Step 400, Total Loss: 30390.216796875
Step 500, Total Loss: 28227.56640625
Step 600, Total Loss: 26691.21875
Step 700, Total Loss: 25532.28515625
Final Stylized Image:
```